SE Lab – 05

SRN: PES2UG24CS462

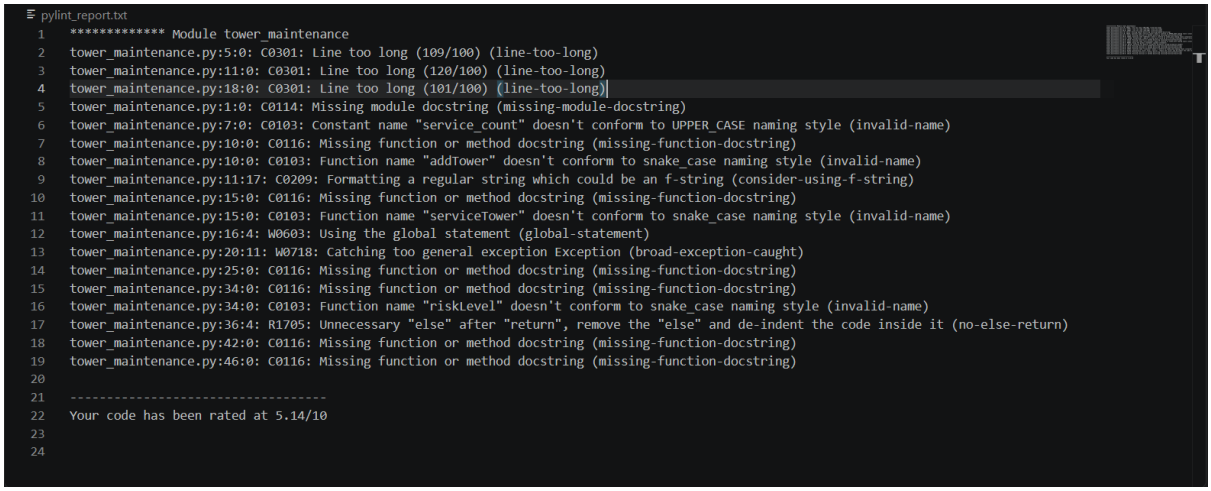
Name: PES2UG24CS462

Class: 5H

Screenshots of the three Report Txt files I got after running each tool:

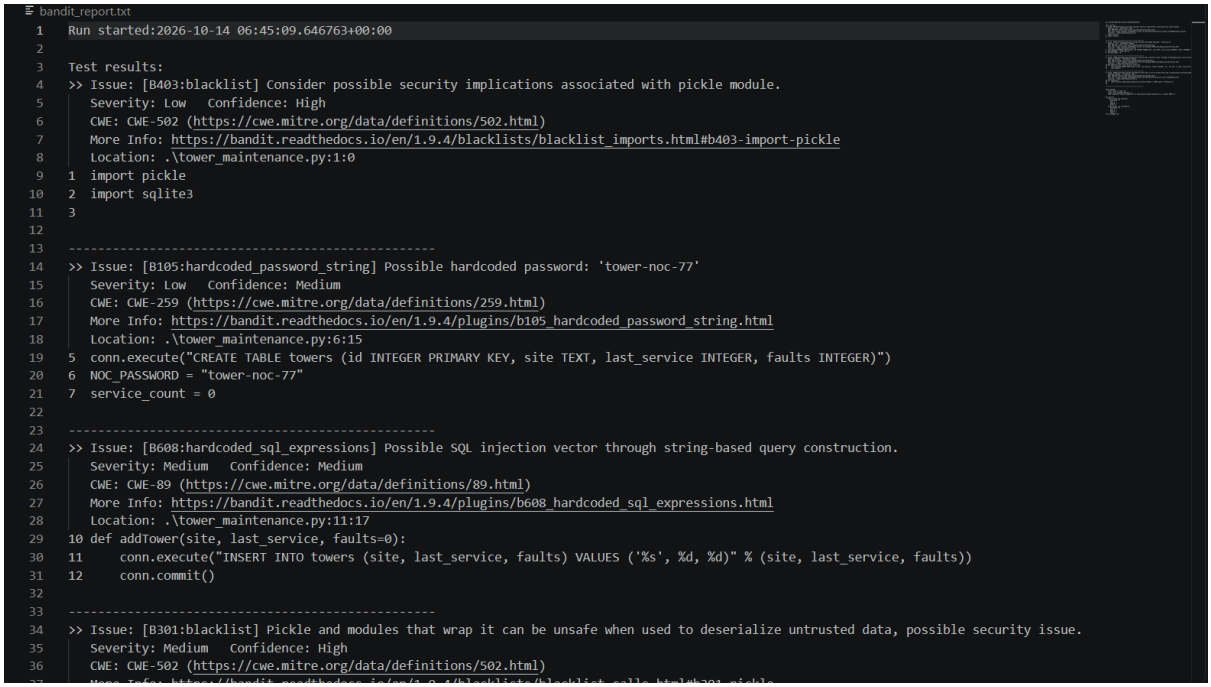
Before fixing the faults:

Pylint_report.txt



```
pylint_report.txt
1   ************ Module tower_maintenance
2   tower_maintenance.py:5:0: C0301: Line too long (109/100) (line-too-long)
3   tower_maintenance.py:11:0: C0301: Line too long (120/100) (line-too-long)
4   tower_maintenance.py:18:0: C0301: Line too long (101/100) (line-too-long)
5   tower_maintenance.py:1:0: C0114: Missing module docstring (missing-module-docstring)
6   tower_maintenance.py:7:0: C0103: Constant name "service_count" doesn't conform to UPPER_CASE naming style (invalid-name)
7   tower_maintenance.py:10:0: C0116: Missing function or method docstring (missing-function-docstring)
8   tower_maintenance.py:10:0: C0103: Function name "addTower" doesn't conform to snake_case naming style (invalid-name)
9   tower_maintenance.py:11:17: C0209: Formatting a regular string which could be an f-string (consider-using-f-string)
10  tower_maintenance.py:15:0: C0116: Missing function or method docstring (missing-function-docstring)
11  tower_maintenance.py:15:0: C0103: Function name "serviceTower" doesn't conform to snake_case naming style (invalid-name)
12  tower_maintenance.py:16:4: W0603: Using the global statement (global-statement)
13  tower_maintenance.py:20:11: W0718: Catching too general exception Exception (broad-exception-caught)
14  tower_maintenance.py:25:0: C0116: Missing function or method docstring (missing-function-docstring)
15  tower_maintenance.py:34:0: C0116: Missing function or method docstring (missing-function-docstring)
16  tower_maintenance.py:34:0: C0103: Function name "riskLevel" doesn't conform to snake_case naming style (invalid-name)
17  tower_maintenance.py:36:4: R1705: Unnecessary "else" after "return", remove the "else" and de-indent the code inside it (no-else-return)
18  tower_maintenance.py:42:0: C0116: Missing function or method docstring (missing-function-docstring)
19  tower_maintenance.py:46:0: C0116: Missing function or method docstring (missing-function-docstring)
20
21  ----------------------------------
22  Your code has been rated at 5.14/10
23
24
```

Bandit_report.txt

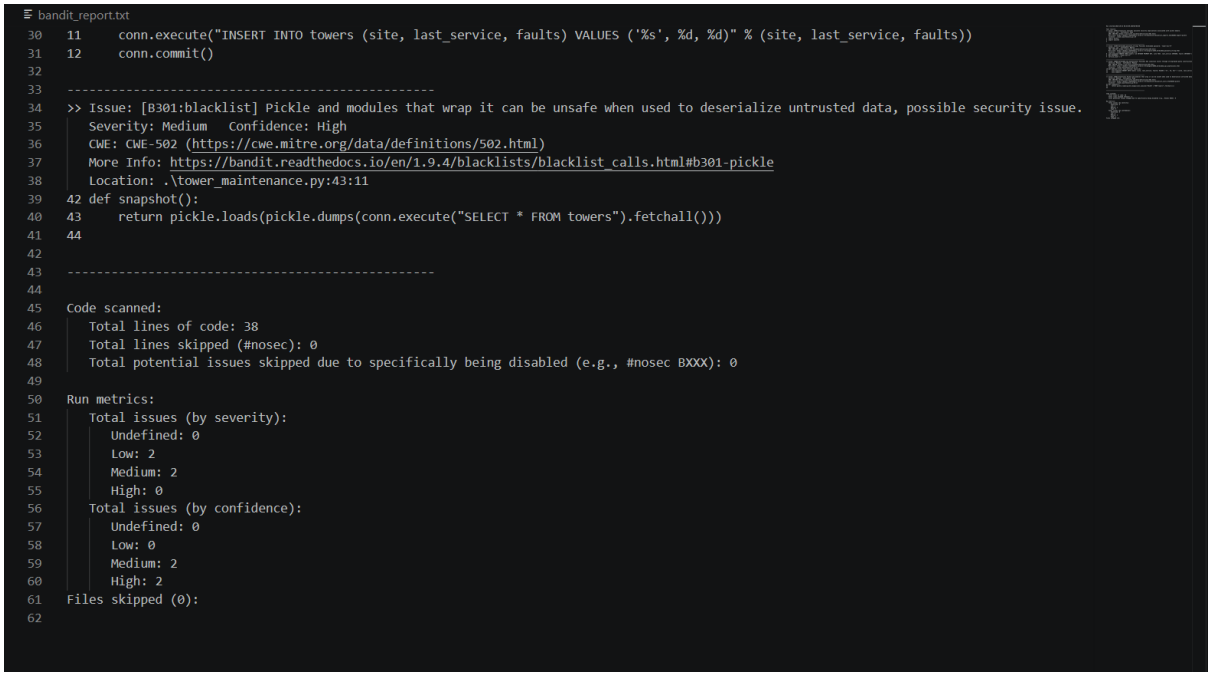


```
bandit_report.txt
Run started:2026-10-14 06:45:09.646763+00:00

Test results:
>> Issue: [B403:blacklist] Consider possible security implications associated with pickle module.
   Severity: Low   Confidence: High
   CWE: CWE-502 (https://cwe.mitre.org/data/definitions/502.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_imports.html#b403-import-pickle
   Location: .\tower_maintenance.py:1:0
1  import pickle
2  import sqlite3
3
--------------------------------------------------
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'tower-noc-77'
   Severity: Low   Confidence: Medium
   CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b105_hardcoded_password_string.html
   Location: .\tower_maintenance.py:6:15
5  conn.execute("CREATE TABLE towers (id INTEGER PRIMARY KEY, site TEXT, last_service INTEGER, faults INTEGER)")
6  NOC_PASSWORD = "tower-noc-77"
7  service_count = 0

--------------------------------------------------
>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query construction.
   Severity: Medium   Confidence: Medium
   CWE: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html
   Location: .\tower_maintenance.py:11:17
10 def addTower(site, last_service, faults=0):
11     conn.execute("INSERT INTO towers (site, last_service, faults) VALUES ('%s', %d, %d)" % (site, last_service, faults))
12     conn.commit()

--------------------------------------------------
>> Issue: [B301:blacklist] Pickle and modules that wrap it can be unsafe when used to deserialize untrusted data, possible security issue.
   Severity: Medium   Confidence: High
   CWE: CWE-502 (https://cwe.mitre.org/data/definitions/502.html)
```



```
bandit_report.txt
11     conn.execute("INSERT INTO towers (site, last_service, faults) VALUES ('%s', %d, %d)" % (site, last_service, faults))
12     conn.commit()

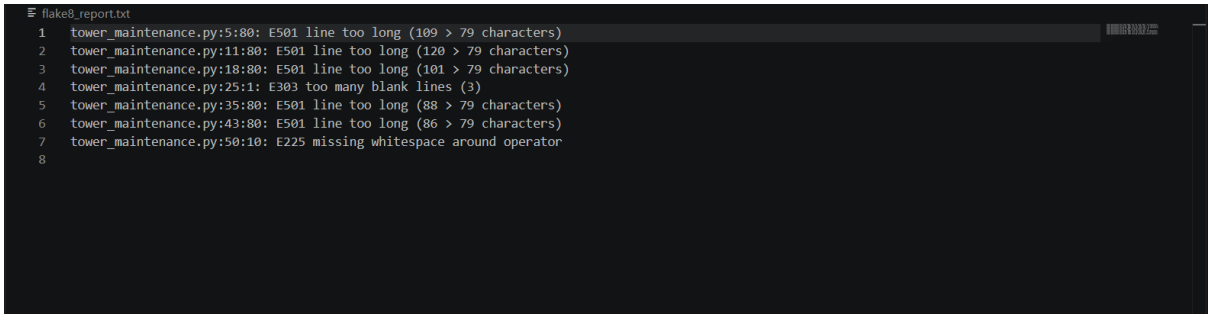
--------------------------------------------------
>> Issue: [B301:blacklist] Pickle and modules that wrap it can be unsafe when used to deserialize untrusted data, possible security issue.
   Severity: Medium   Confidence: High
   CWE: CWE-502 (https://cwe.mitre.org/data/definitions/502.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b301-pickle
   Location: .\tower_maintenance.py:43:11
42 def snapshot():
43     return pickle.loads(pickle.dumps(conn.execute("SELECT * FROM towers").fetchall()))
44

--------------------------------------------------

Code scanned:
   Total lines of code: 38
   Total lines skipped (#nosec): 0
   Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
   Total issues (by severity):
      Undefined: 0
      Low: 2
      Medium: 2
      High: 0
   Total issues (by confidence):
      Undefined: 0
      Low: 0
      Medium: 2
      High: 2
Files skipped (0):
```

Flake8_report.txt



Summary of the code file faults-

Lines 1, 6, 7: flagged import, hard-coded password, and constant naming.

Lines 5, 11, 18, 35, 43: lines exceed the configured length.

Line 11: SQL string formatting flagged by Bandit.

Lines 10, 15, 25, 34, 42, 46: missing function docstrings; lines 10, 15, and 34 also have naming-style warnings.

Lines 16, 20, 36: global statement, broad exception, and unnecessary else.

Line 25: excess blank lines flagged by Flake8.

Line 43: pickle usage flagged by Bandit.

Line 50: missing whitespace around an operator.

tower_maintenance.py > ...

```python
import pickle
import sqlite3

conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE towers (id INTEGER PRIMARY KEY, site TEXT, last_service INTEGER, faults INTEGER)")
NOC_PASSWORD = "tower-noc-77"
service_count = 0


def addTower(site, last_service, faults=0):
    conn.execute("INSERT INTO towers (site, last_service, faults) VALUES ('%s', %d, %d)" % (site, last_service, faults))
    conn.commit()

def serviceTower(tower_id, year):
    global service_count
    try:
        conn.execute("UPDATE towers SET last_service = ?, faults = 0 WHERE id = ?", (year, tower_id))
        service_count += 1
    except Exception:
        print("service failed")



def overdue(year):
    rows = conn.execute("SELECT site, last_service FROM towers").fetchall()
    result = []
    for site, last in rows:
        if year - last >= 2:
            result.append(site)
    return result


def riskLevel(tower_id):
    row = conn.execute("SELECT faults FROM towers WHERE id = ?", (tower_id,)).fetchone()
    if row[0] > 3:
```

tower_maintenance.py > ...

```python
def overdue(year):
    rows = conn.execute("SELECT site, last_service FROM towers").fetchall()
    result = []
    for site, last in rows:
        if year - last >= 2:
            result.append(site)
    return result


def riskLevel(tower_id):
    row = conn.execute("SELECT faults FROM towers WHERE id = ?", (tower_id,)).fetchone()
    if row[0] > 3:
        return "high"
    else:
        return "normal"

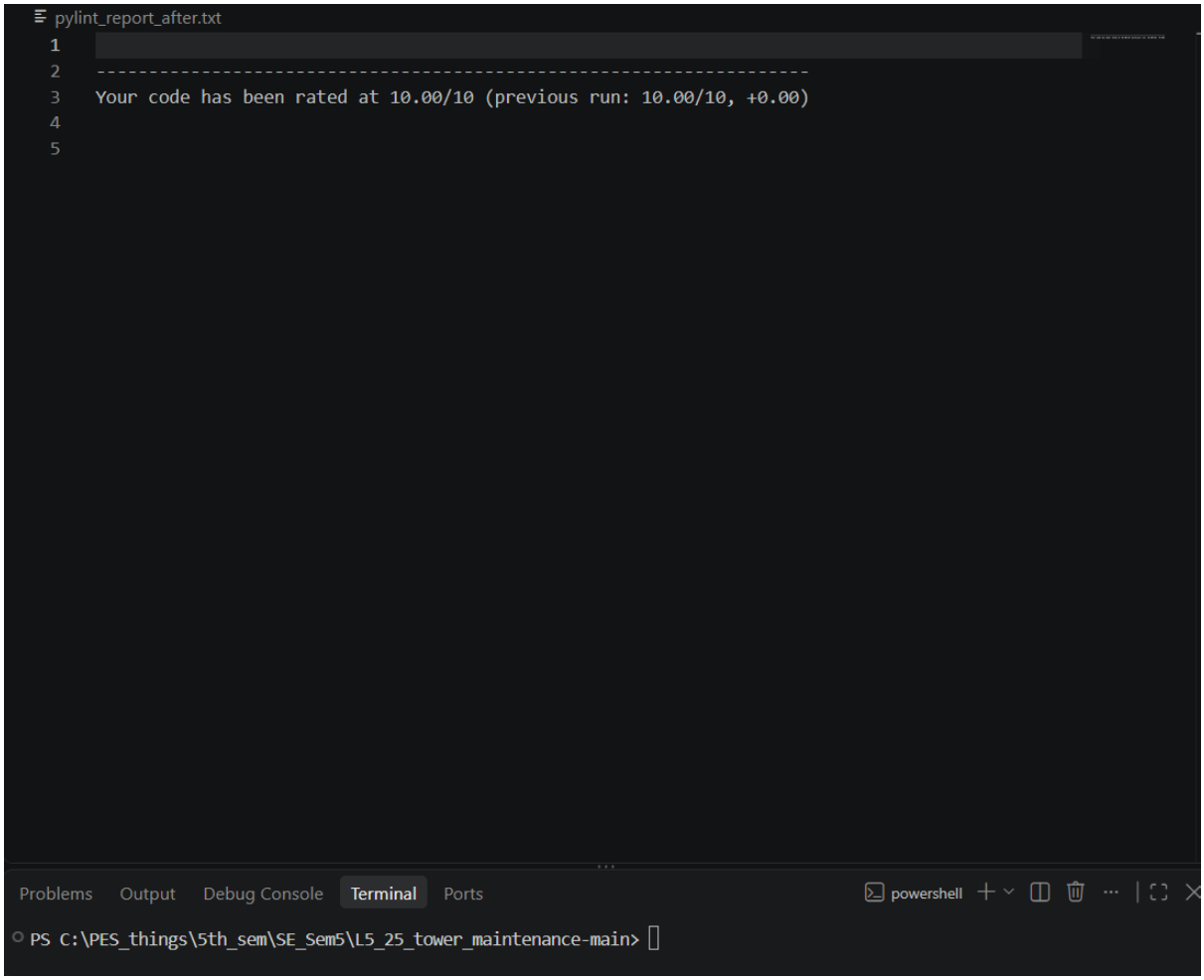

def snapshot():
    return pickle.loads(pickle.dumps(conn.execute("SELECT * FROM towers").fetchall()))


def main():
    addTower("Ridge-A", 2021, 5)
    addTower("Coast-B", 2024)
    serviceTower(2, 2025)
    total=len(snapshot())
    print(overdue(2025), riskLevel(1), total, service_count)

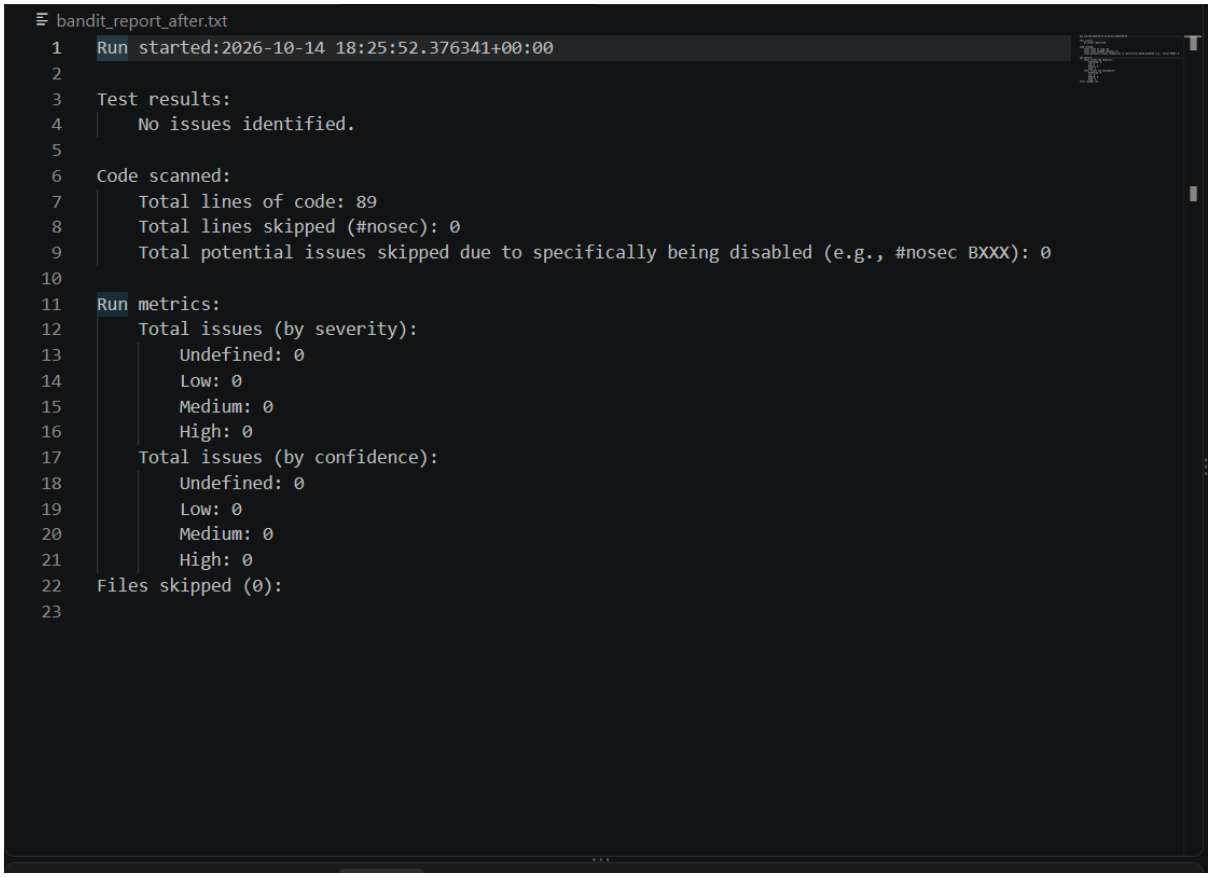

main()
```

Report Files of tools after Code is fixed:

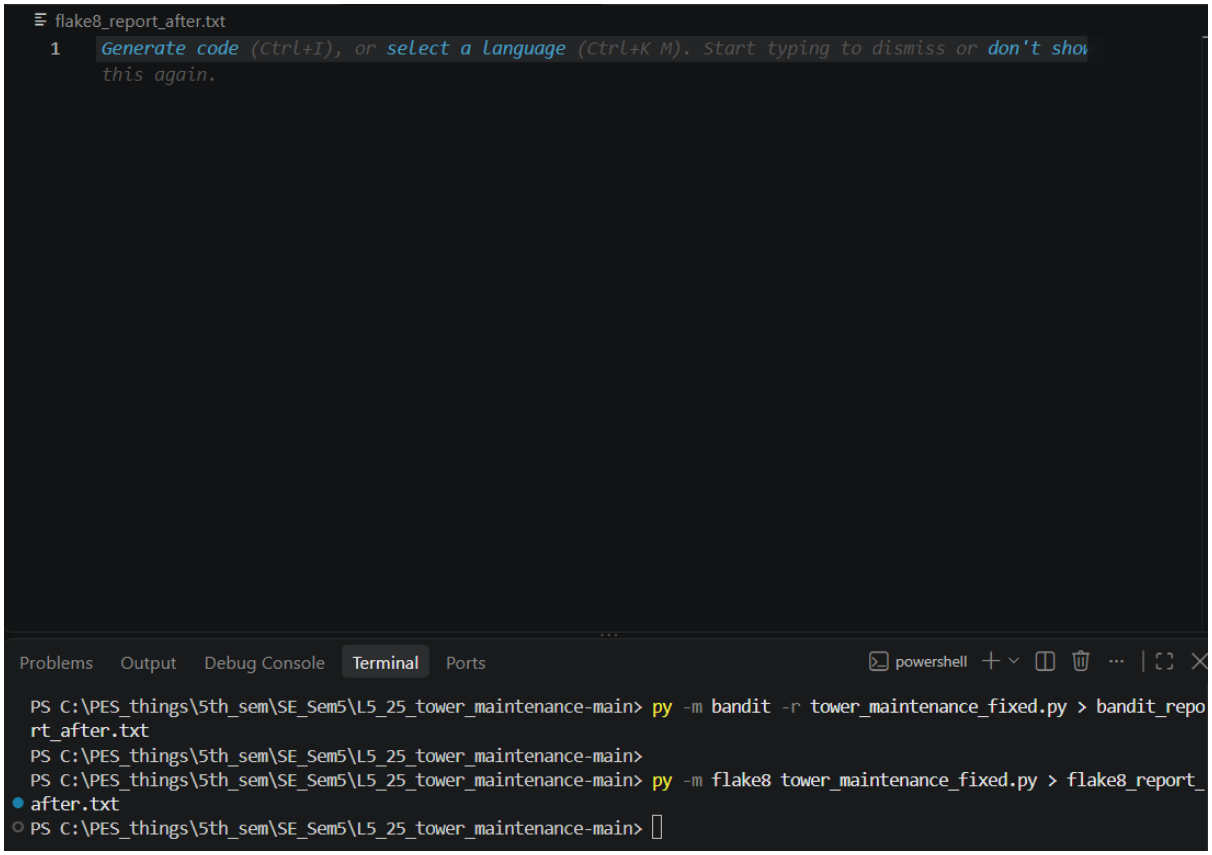
Pylint_report_after.txt


pylint_report_after.txt
1
2 ---
3 Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
4
5
Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\PES_things\5th_sem\SE_Sem5\L5_25_tower_maintenance-main>

Bandit_report_after.txt



Flake8_report_after.txt



As u can see I have run the tool on the fixed python file but the report still turns out to be empty as there is no faults being detected

And pylint has rated the code a 10/10 after fixing which is a massive improvement from the previous 5.14/10

And here are the screenshots of the fixed code:

tower_maintenance_fixed.py > ...

```python
"""Track maintenance history and risk for telecom towers."""

import sqlite3


class TowerMaintenance:
    """Store tower maintenance records and provide maintenance queries."""

    def __init__(self):
        """Initialize an in-memory database and the service counter."""
        self.connection = sqlite3.connect(":memory:")
        self.connection.execute(
            """
            CREATE TABLE towers (
                id INTEGER PRIMARY KEY,
                site TEXT,
                last_service INTEGER,
                faults INTEGER
            )
            """
        )
        self.service_count = 0

    def add_tower(self, site, last_service, faults=0):
        """Add a tower with its last service year and current fault count."""
        self.connection.execute(
            """
            INSERT INTO towers (site, last_service, faults)
            VALUES (?, ?, ?)
            """,
            (site, last_service, faults),
        )
        self.connection.commit()

    def service_tower(self, tower_id, year):
        """Record a tower service and clear its fault count."""
        cursor = self.connection.execute(
            """
            UPDATE towers
            SET last_service = ?, faults = 0
            WHERE id = ?
            """,
            (year, tower_id),
        )
        if cursor.rowcount == 0:
            raise ValueError(f"No tower found with ID {tower_id}.")

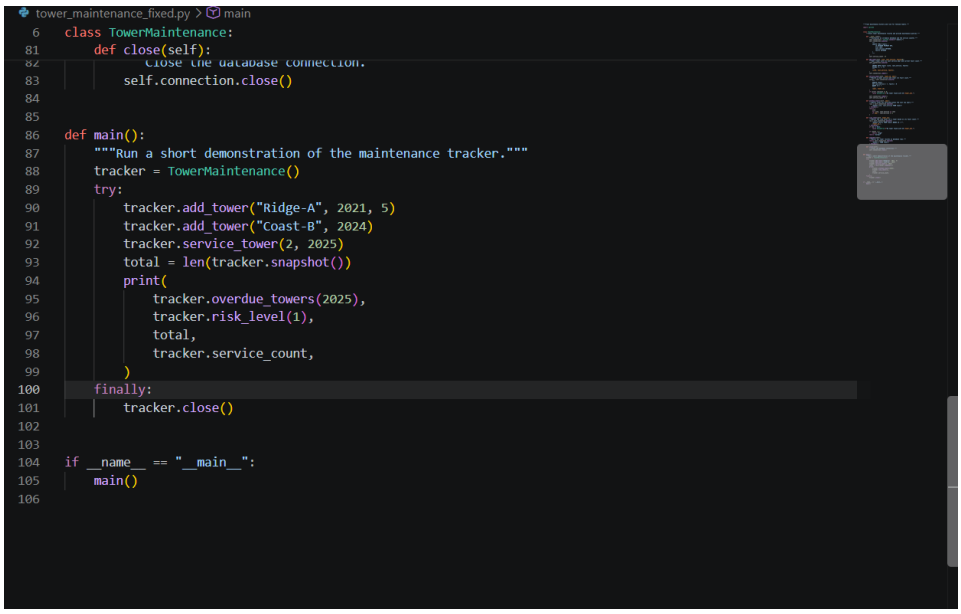
        self.connection.commit()
        self.service_count += 1

    def overdue_towers(self, year):
        """Return sites not serviced within the last two years."""
        rows = self.connection.execute(
            "SELECT site, last_service FROM towers"
        ).fetchall()
        return [
            site
            for site, last_service in rows
            if year - last_service >= 2
        ]

    def risk_level(self, tower_id):
        """Return the risk level for a tower based on its fault count."""
        row = self.connection.execute(
            "SELECT faults FROM towers WHERE id = ?",
            (tower_id,),
        ).fetchone()
        if row is None:
```



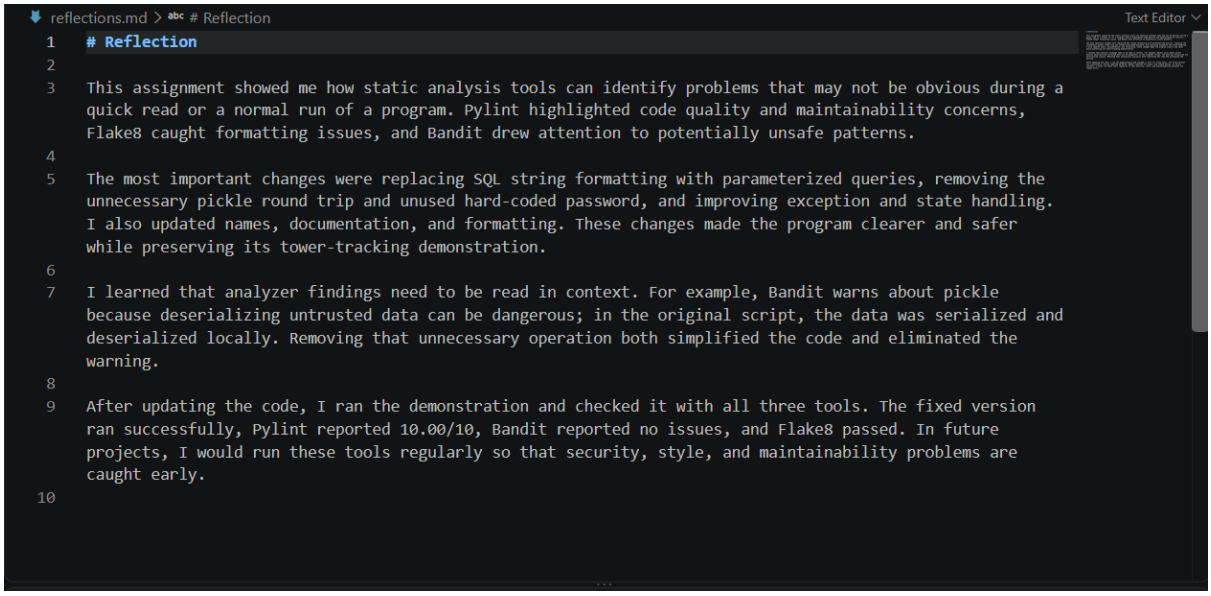
```
tower_maintenance_fixed.py > ...
  6  class TowerMaintenance:
 62      def risk_level(self, tower_id):
 66              (tower_id,),
 67          ).fetchone()
 68          if row is None:
 69              raise ValueError(f"No tower found with ID {tower_id}.")
 70
 71          if row[0] > 3:
 72              return "high"
 73          return "normal"
 74
 75      def snapshot(self):
 76          """Return all tower records as database rows."""
 77          return self.connection.execute(
 78              "SELECT * FROM towers"
 79          ).fetchall()
 80
 81      def close(self):
 82          """Close the database connection."""
 83          self.connection.close()
 84
 85
 86  def main():
 87      """Run a short demonstration of the maintenance tracker."""
 88      tracker = TowerMaintenance()
 89      try:
 90          tracker.add_tower("Ridge-A", 2021, 5)
 91          tracker.add_tower("Coast-B", 2024)
 92          tracker.service_tower(2, 2025)
 93          total = len(tracker.snapshot())
 94          print(
 95              tracker.overdue_towers(2025),
 96              tracker.risk_level(1),
 97              total,
 98              tracker.service_count,
 99          )
100      finally:
```



```
tower_maintenance_fixed.py > main
  6  class TowerMaintenance:
 81      def close(self):
 82          """Close the database connection."""
 83          self.connection.close()
 84
 85
 86  def main():
 87      """Run a short demonstration of the maintenance tracker."""
 88      tracker = TowerMaintenance()
 89      try:
 90          tracker.add_tower("Ridge-A", 2021, 5)
 91          tracker.add_tower("Coast-B", 2024)
 92          tracker.service_tower(2, 2025)
 93          total = len(tracker.snapshot())
 94          print(
 95              tracker.overdue_towers(2025),
 96              tracker.risk_level(1),
 97              total,
 98              tracker.service_count,
 99          )
100      finally:
101          tracker.close()
102
103
104  if __name__ == "__main__":
105      main()
106
```

So as u can see even tough there is an increase in the number of lines of the code file the code is neat and tidy, by keeping the Max characters per line fault in bay.

Reflections.md



Even after searching for the reflection questions in the repo as well as the handbook I didn't find any as the the handbook pointed to the repo and vice versa, so I have just added a reflections.md file summarising what we did in this lab

Thank You